

TASK –3

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No: 3

Student Name: Reddy Mohini

Roll No.: A24126552110

Branch: CSM

Section: B

Date:

1. TITLE

JavaScript-Based To-Do List Application

2. AIM

To create a JavaScript-based To-Do List application using DOM manipulation, dynamic HTML element creation, and event handling.

3. OBJECTIVE

- To understand DOM selection and manipulation.
- To dynamically create and manage HTML elements using JavaScript.
- To implement event handling for interactive UI updates.
- To apply CSS styling for a clean, card-based layout.
- To validate user input and manage empty states.

4. THEORY

- DOM Manipulation allows dynamic creation, modification, and deletion of HTML elements.
- Event Handling enables interaction with user actions (e.g., button clicks).
- Methods like `createElement()`, `appendChild()`, and `classList.toggle()` are used to manage tasks.
- CSS styling improves user experience with visual feedback (completed tasks, hover effects, empty states).

Flow of application: User Input → DOM Manipulation → Task Creation → Event Handling → Interactive To-Do List

5. ALGORITHM / PROCEDURE

1. Create an HTML file with input field and “Add Task” button.
2. Add a container for task list and empty message.
3. Capture input using `getElementById()`.
4. On button click, validate input.
5. If valid, create a new task element dynamically.
6. Add “Complete” and “Delete” buttons to each task.
7. Implement event handling for task completion and deletion.
8. Update empty message visibility based on task list.
9. Apply CSS styling for layout and feedback.
10. Test with multiple tasks.

6. PROGRAM

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>My To-Do List</title>

<style>

    /* Page styling */
    body {
        font-family: Arial, sans-serif;
        background-color: #f2f2f2;
        margin: 0;
        padding: 40px;
    }

    /* Main container */
    .container {
        width: 500px;
        margin: auto;
        background-color: white;
        padding: 25px;
        border-radius: 10px;
        box-shadow: 0 0 10px gray;
    }

    /* Heading */
    h1 {
        text-align: center;
    }

    /* Input box */
    #taskInput {
        width: 65%;
        padding: 10px;
        border: 1px solid #ccc;
        border-radius: 5px;
        font-size: 16px;
    }

    /* Add button */
    #addBtn {
        padding: 10px 15px;
        background-color: green;
        color: white;
    }
```

```
        border: none;
        border-radius: 5px;
        cursor: pointer;
    }

    /* Task list */
    #taskList {
        margin-top: 20px;
        padding: 0;
    }

    /* Individual task */
    .task {
        display: flex;
        align-items: center;
        justify-content: space-between;

        padding: 12px;
        margin-bottom: 10px;

        background-color: #f8f8f8;

        border: 1px solid #ddd;
        border-radius: 5px;
    }

    /* Task text */
    .task-text {
        flex: 1;
        font-size: 16px;
    }

    /* Completed task */
    .completed {
        text-decoration: line-through;
        color: gray;
    }

    /* Buttons inside task */
    .complete-btn,
    .delete-btn {
        padding: 7px 10px;
        margin-left: 5px;
    }
```

```

        border: none;
        border-radius: 4px;
        cursor: pointer;
    }

    .complete-btn {
        background-color: blue;
        color: white;
    }

    .delete-btn {
        background-color: red;
        color: white;
    }

    /* Empty message */
    #emptyMessage {
        text-align: center;
        color: gray;
        margin-top: 20px;
    }

</style>

</head>

<body>

    <div class="container">

        <h1>My To-Do List</h1>

        <!-- Input and Add button -->

        <input
            type="text"
            id="taskInput"
            placeholder="Enter a task....."
        >

        <button id="addBtn">
            Add Task

```

```
</button>

<!-- Empty message -->

<p id="emptyMessage">
  No tasks available.
</p>

<!-- Task list -->

<div id="taskList"></div>

</div>

<script>

  // Select the required HTML elements

  const taskInput = document.getElementById("taskInput");

  const addBtn = document.getElementById("addBtn");

  const taskList = document.getElementById("taskList");

  const emptyMessage =
    document.getElementById("emptyMessage");

  // Function to update the empty message

  function updateEmptyMessage() {

    if (taskList.children.length === 0) {

      emptyMessage.style.display = "block";

    } else {

      emptyMessage.style.display = "none";

    }

  }

}
```

```
}

// Add Task button click event

addBtn.addEventListener("click", function() {

    // Get the task entered by the user

    const taskText = taskInput.value.trim();

    // Check if the input is empty

    if (taskText === "") {

        alert("Please enter a task.");

        return;

    }

    // Create the main task container

    const task = document.createElement("div");

    task.className = "task";

    // Create the task text

    const taskTextElement =
        document.createElement("span");

    taskTextElement.className = "task-text";

    taskTextElement.textContent = taskText;

    // Create Complete button

    const completeBtn =
        document.createElement("button");
```

```
completeBtn.className = "complete-btn";

completeBtn.textContent = "Complete";

// Create Delete button

const deleteBtn =
    document.createElement("button");

deleteBtn.className = "delete-btn";

deleteBtn.textContent = "Delete";

// Complete button event

completeBtn.addEventListener("click", function() {

    taskTextElement.classList.toggle("completed");

});

// Delete button event

deleteBtn.addEventListener("click", function() {

    task.remove();

    updateEmptyMessage();

});

// Add elements to task container

task.appendChild(taskTextElement);

task.appendChild(completeBtn);

task.appendChild(deleteBtn);
```



```

        // Add task to task list

        taskList.appendChild(task);

        // Clear input box

        taskInput.value = "";

        // Update empty message

        updateEmptyMessage();

    });

    // Display empty message initially

    updateEmptyMessage();

</script>

</body>

</html>

```

7. CODE EXPLANATION

Code	Explanation
<code>document.getElementById()</code>	Selects HTML elements.
<code>addEventListener("click")</code>	Handles button click events.

<code>createElement()</code>	Dynamically creates new task elements.
<code>appendChild()</code>	Adds elements to the DOM.
<code>classList.toggle("completed")</code>	Toggles strikethrough style for completed tasks.
<code>task.remove()</code>	Deletes a task from the list.
<code>updateEmptyMessage()</code>	Shows/hides empty state message.

8. TEST CASES AND EXECUTION DATA

Test Case	Input	Expected Result	Actual Result	Status
TC-01	Enter "Complete Assignment"	Task added to list	Works correctly	pass
TC-02	Leave input empty	Alert shown	Works correctly	Pass
TC-03	Click "Complete" button	Task text toggles strikethrough	Works correctly	Pass
TC-04	Click "Delete" button	Task removed from list	Works correctly	Pass
TC-05	No tasks present	"No tasks available" message displayed	Works correctly	Pass

9.OUTPUT

My To-Do List

Add Task

No tasks available.

My To-Do List

Add Task

complete my project

Complete

Delete

10.RESULT

Thus, a **JavaScript-based To-Do List Application** was successfully developed. The application demonstrates DOM manipulation, dynamic HTML element creation, event handling, and interactive UI updates.